by augmenting the given prompt with an additional set of tokens that the LLM conditions on to obtain the modified distribution, or **(2) at the output level**: by sampling multiple candidates from the standard LM and performing surgery on these candidates. In other words, we could either modify the **proposal distribution** induced by the LLM itself such that it is an improvement over naïvely conditioning on the prompt or we could use some post-hoc **verifiers or scorers** to perform output modifications. This process is reminiscent of Markov chain Monte Carlo (MCMC) [2] sampling from a complex target distribution but by combining a simple proposal distribution and a score function. Modifying the proposal distribution directly by altering input tokens and using a verifier form two independent axes of our study.

Modifying the proposal distribution. One way to improve the proposal distribution is to directly optimize the model for a given reasoning task via RL-inspired finetuning methods such as STaR or ReST^{EM} [35, 50]. Note that these techniques do not utilize any additional input tokens but specifically finetune the model to induce an improved proposal distribution. Instead, techniques such as self-critique [4, 8, 23, 30] enable the model itself to improve its own proposal distribution at test time by instructing it to critique and revise its own outputs in an iterative fashion. Since prompting off-the-shelf models is not effective at enabling effective revisions at test time, we specifically finetune models to iteratively revise their answers in complex reasoning-based settings. To do so, we utilize the approach of finetuning on on-policy data with Best-of-N guided improvements to the model response [28].

Optimizing the verifier. In our abstraction of the proposal distribution and verifier, the verifier is used to aggregate or select the best answer from the proposal distribution. The most canonical way to use such a verifier is by applying best-of-N sampling, wherein we sample N complete solutions and then select the best one according to a verifier [7]. However, this approach can be further improved by training a process-based verifier [22], or a process reward model (PRM), which produces a prediction of the correctness of each intermediate step in an solution, rather than just the final answer. We can then utilize these per-step predictions to perform tree search over the space of solutions, enabling a potentially more efficient and effective way to search against a verifier, compared to naïve best-of-N [6, 10, 48].

3. How to Scale Test-Time Computation Optimally

Given the unification of various methods, we would now like to understand how to *most effectively* utilize test-time computation to improve LM performance on a given prompt. Concretely we wish to answer:

Problem setup

We are given a prompt and a test-time compute budget within which to solve the problem. Under the abstraction above, there are different ways to utilize test-time computation. Each of these methods may be more or less effective depending on the specific problem given. How can we determine the *most effective* way to utilize test-time compute for a given prompt? And how well would this do against simply utilizing a much bigger pretrained model?

When either refining the proposal distribution or searching against a verifier, there are several different hyper-parameters that can be adjusted to determine how a test-time compute budget should be allocated. For example, when using a model finetuned for revisions as the proposal distribution and an ORM as the verifier, we could either spend the full test-time compute budget on generating N independent samples in parallel from the model and then apply best-of-N, or we could sample N revisions in sequence using a revision model and then select the best answer in the sequence with an ORM, or strike a balance between these extremes. Intuitively, we might expect "easier" problems to benefit more from revisions, since the

model's initial samples are more likely to be on the right track but may just need further refinement. On the other hand, challenging problems may require more exploration of different high-level problem solving strategies, so sampling many times independently in parallel may be preferable in this setting.

In the case of verifiers, we also have the option to choose between different search algorithms (e.g. beam-search, lookahead-search, best-of-N), each of which may exhibit different properties depending on the quality of the verifier and proposal distribution at hand. More sophisticated search procedures might be more useful in harder problems compared to a much simpler best-of-N or majority baseline.

3.1. Test-Time Compute-Optimal Scaling Strategy

In general, we would therefore like to select the *optimal* allocation of our test-time compute budget for a given problem. To this end, for any given approach of utilizing test-time compute (e.g., revisions and search against a verifier in this paper, various other methods elsewhere), we define the **"test-time compute-optimal scaling strategy"** as the strategy that chooses hyperparameters corresponding to a given test-time strategy for maximal performance benefits on a given prompt at test time. Formally, define $\text{Target}(\theta, N, q)$ as the distribution over natural language output tokens induced by the model for a given prompt q, using test-time compute hyper-parameters θ, and a compute budget of N. We would like to select the hyper-parameters θ which maximize the accuracy of the target distribution for a given problem. We express this formally as:

$$\theta^*_{q,a^*(q)}(N) = \text{argmax}_\theta \left(\mathbb{E}_{y \sim \text{Target}(\theta,N,q)} \left[\mathbb{1}_{y=y^*(q)} \right] \right), \tag{1}$$

where $y^*(q)$ denotes the ground-truth correct response for q, and $\theta^*_{q,y^*(q)}(N)$ represents the test-time compute-optimal scaling strategy for the problem q with compute budget N.

3.2. Estimating Question Difficulty for Compute-Optimal Scaling

In order to effectively analyze the test-time scaling properties of the different mechanisms discussed in Section 2 (e.g. the proposal distribution and the verifier), we will prescribe an approximation to this optimal strategy $\theta^*_{q,y^*(q)}(N)$ as a function of a statistic of a given prompt. This statistic estimates a notion of *difficulty* for a given prompt. The compute-optimal strategy is defined as a function of the difficulty of this prompt. Despite being only an approximate solution to the problem shown in Equation 1, we find that it can still induce substantial improvements in performance over a baseline strategy of allocating this inference-time compute in an ad-hoc or uniformly-sampled manner.

Our estimate of the question difficulty assigns a given question to one of five difficulty levels. We can then use this discrete difficulty categorization to estimate $\theta^*_{q,y^*(q)}(N)$ on a validation set for a given test-time compute budget. We then apply these compute-optimal strategies on the test-set. Concretely, we select the best performing test-time compute strategy for each difficulty bin independently. In this way, question difficulty acts as a sufficient statistic of a question when designing the compute-optimal strategy.

Defining difficulty of a problem. Following the approach of Lightman et al. [22], we define question difficulty as a function of a given base LLM. Specifically, we bin the model's pass@1 rate – estimated from 2048 samples – on each question in the test set into five quantiles, each corresponding to increasing difficulty levels. We found this notion of model-specific difficulty bins to be more predictive of the efficacy of using test-time compute in contrast to the hand-labeled difficulty bins in the MATH dataset.

That said, we do note that assessing a question's difficulty as described above assumes oracle access to a ground-truth correctness checking function, which is of course not available upon deployment where we

are only given access to test prompts that we don't know the answer to. In order to be feasible in practice, a compute-optimal scaling strategy conditioned on difficulty needs to first assess difficulty and then utilize the right scaling strategy to solve this problem. Therefore, we approximate the problem's difficulty via a **model-predicted notion of difficulty**, which performs the same binning procedure over the the averaged final answer score from a learned verifier (and not groundtruth answer correctness checks) on the same set of 2048 samples per problem. We refer to this setting as **model-predicted difficulty** and the setting which relies on the ground-truth correctness as **oracle difficulty**.

While model-predicted difficulty removes the need for need knowing the ground truth label, estimating difficulty in this way still incurs additional computation cost during inference. That said, this one-time inference cost can be subsumed within the cost for actually running an inference-time strategy (e.g., when using a verifier, one could use the same inference computation for also running search). More generally, this is akin to exploration-exploitation tradeoff in reinforcement learning: in actual deployment conditions, we must balance the compute spent in assessing difficulty vs applying the most compute-optimal approach. This is a crucial avenue for future work (see Section 8) and our experiments do not account for this cost largely for simplicity, since our goal is to present some of the first results of *what is in fact possible* by effectively allocating test-time compute.

So as to avoid confounders with using the same test set for computing difficulty bins and for selecting the compute-optimal strategy, we use two-fold cross validation on each difficulty bin in the test set. We select the best-performing strategy according to performance on one fold and then measure performance using that strategy on the other fold and vice versa, averaging the results of the two test folds.

4. Experimental Setup

We first outline our experimental setup for conducting this analysis with multiple verifier design choices and proposal distributions, followed by the analysis results in the subsequent sections.

Datasets. We expect test-time compute to be most helpful when models already have all the basic "knowledge" needed to answer a question, and instead the primary challenge is about drawing (complex) inferences from this knowledge. To this end, we focus on the MATH [13] benchmark, which consists of high-school competition level math problems with a range of difficulty levels. For all experiments, we use the dataset split consisting of 12k train and 500 test questions, used in Lightman et al. [22].

Models. We conduct our analysis using the PaLM 2-S* [3] (Codey) base model. We believe this model is representative of the capabilities of many contemporary LLMs, and therefore think that our findings likely transfer to similar models. Most importantly, this model attains a non-trivial performance on MATH and yet has not saturated, so we expect this model to provide a good test-bed for us.

5. Scaling Test-Time Compute via Verifiers

In this section we analyze how test-time compute can be scaled by optimizing a verifier, as effectively as possible. To this end, we study different approaches for performing test-time search with process verifiers (PRMs) and analyze the test-time compute scaling properties of these different approaches.

5.1. Training Verifiers Amenable to Search

PRM training. Originally PRM training [22, 42] used human crowd-worker labels. While Lightman et al. [22] released their PRM training data (i.e., the PRM800k dataset), we found this data to be largely